Winter Semester 2026

CSE401-Database Management System

Project title: Startup Collaboration Platform

Group 20

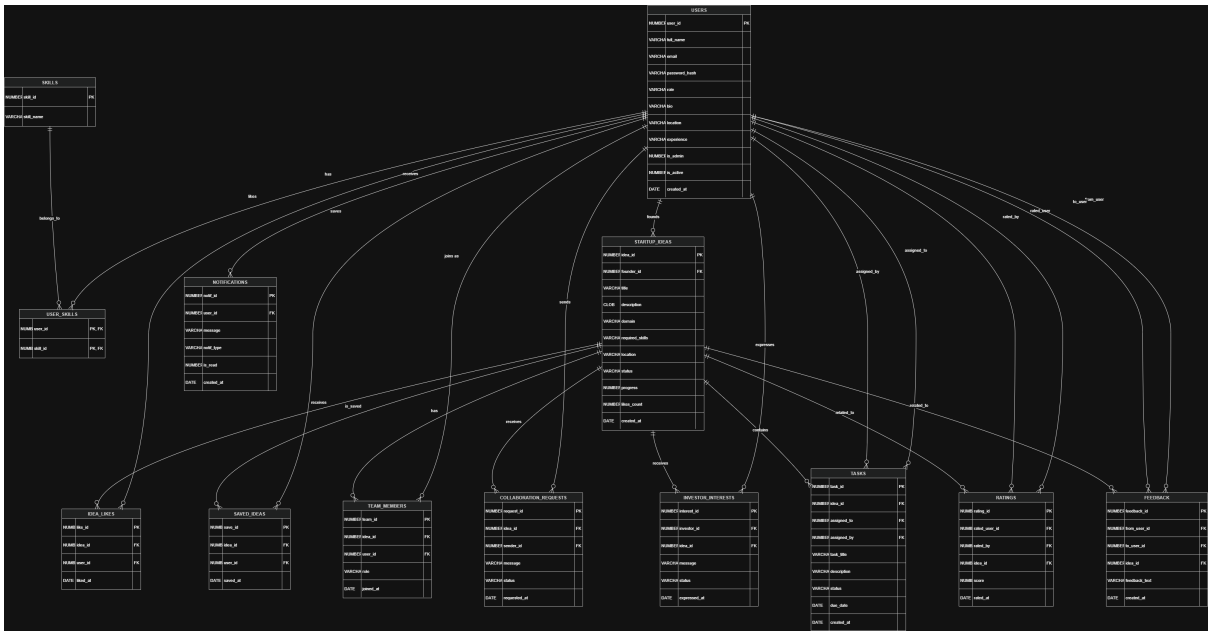
AU2340258 Harsh Patel
AU2340194 Hevit Makavana
AU2340087 Maanya Patel
AU2340055 Krissa Gandhi

1. Description of the Project (System Requirement Specification)

The Startup Collaboration Platform bridges the gap among investors, creators, developers/professionals, and founders in an online environment. Its basic functionalities are:

- **Profiles and User Authentication:** Users can register and become founders. Developers and investors can also create accounts. Users can update their skills, location, and bio in their profiles.
- **Startup Concepts Management:** Founders can generate and manage new business ideas, specify required skills and domain, and track progress through task completion.
- **Skills Matching Algorithm:** The system identifies potential team candidates and matches users based on required skills to provide suitable suggestions.
- **Teamwork & Collaboration:** Professionals can request to participate in startup ideas, and founders can approve or disapprove applications.
- **Management of Tasks:** Tasks can be assigned by founders and team members, and progress is automatically updated based on completed tasks.
- **Reaching Out to Investors:** Investors can search concepts by domain, show interest in funding opportunities, and founders are notified about investor actions.
- **Feedback and Evaluation Mechanism:** Collaborators can rate each other, ensuring transparency and trust through feedback.
- **Notification Engine:** Users are notified about investor interests, task assignments, and join requests, implemented using PL/SQL triggers.

2. Entity-Relationship Diagram



Link to the ER - Diagram

Figure 1: Entity Relationship Diagram of the Platform

3. Table Design (Data Dictionary / Relational Model)

1. USERS Table

Column Name	Data Type	Constraints
user_id	NUMBER	PRIMARY KEY
full_name	VARCHAR2(100)	NOT NULL
email	VARCHAR2(150)	UNIQUE, NOT NULL
password_hash	VARCHAR2(255)	NOT NULL
role	VARCHAR2(20)	CHECK (Founder, Developer, Investor)
is_active	NUMBER(1)	DEFAULT 1
created_at	DATE	DEFAULT SYSDATE

2. STARTUP_IDEAS Table

Column Name	Data Type	Constraints
idea_id	NUMBER	PRIMARY KEY
founder_id	NUMBER	FOREIGN KEY (users) ON DELETE CASCADE
title	VARCHAR2(200)	NOT NULL
domain	VARCHAR2(50)	CHECK (AI, FinTech, SaaS, etc.)
progress	NUMBER(3)	DEFAULT 0, CHECK (0-100)

3. TASKS Table

Column Name	Data Type	Constraints
task_id	NUMBER	PRIMARY KEY
idea_id	NUMBER	FOREIGN KEY (startup_ideas)
assigned_to	NUMBER	FOREIGN KEY (users)
status	VARCHAR2(20)	CHECK (Pending, In Progress, Completed)

4. Code to Connect Front-End and Back-End

Back-End Connection Setup (Python Flask with OracleDB)

```python
import oracledb
from flask import Flask

app = Flask(__name__)

DB_USERNAME = "system"
DB_PASSWORD = "Harsh2005"
DB_DSN      = "127.0.0.1:1521/XE"

def get_db_connection():
    try:
        conn = oracledb.connect(user=DB_USERNAME, password=DB_PASSWORD, dsn=DB_DSN)
        return conn
    except oracledb.Error as e:
        print(f"Database connection error: {e}")
        return None
```

Front-End Fetch Data Setup (JavaScript to Flask API)

```javascript
// Example of front-end logic hitting the API
async function loginUser(email, password) {
    const response = await fetch('http://127.0.0.1:5000/api/login', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ email, password })
    });
    const data = await response.json();
    if(data.status === 'success') {
        localStorage.setItem('user_id', data.user_id);
    }
}
```

5. Stored Procedures, Functions, and Triggers

1. Stored Procedure: User Registration

Oracle PL/SQL:

```
CREATE OR REPLACE PROCEDURE sp_register_user (
    p_full_name     IN VARCHAR2,
    p_email         IN VARCHAR2,
    p_password_hash IN VARCHAR2,
    p_role          IN VARCHAR2,
    p_location      IN VARCHAR2,
    p_result        OUT VARCHAR2
) AS
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count FROM users WHERE email = p_email;
    IF v_count > 0 THEN
        p_result := 'ERROR: Email already registered.';
    ELSE
        INSERT INTO users (user_id, full_name, email, password_hash,
   role, location)
        VALUES (seq_users.NEXTVAL, p_full_name, p_email,
   p_password_hash, p_role, p_location);
        COMMIT;
        p_result := 'SUCCESS: User registered successfully.';
    END IF;
EXCEPTION
    WHEN OTHERS THEN
        p_result := 'ERROR: ' || SQLERRM;
END;
```

Statement to Call from Flask (Back-end called by Front-end):

```
cursor = conn.cursor()
result_var = cursor.var(oracledb.STRING)
cursor.callproc('sp_register_user', [full_name, email, password_hash,
   role, location, result_var])
result = result_var.getvalue() # Returns 'SUCCESS' or 'ERROR'
```

2. Function: Get Average Rating of a User

Oracle PL/SQL:

```
CREATE OR REPLACE FUNCTION fn_get_avg_rating (
    p_user_id IN NUMBER
) RETURN NUMBER AS
    v_avg NUMBER;
BEGIN
    SELECT ROUND(AVG(score), 2) INTO v_avg
    FROM ratings WHERE rated_user_id = p_user_id;
    RETURN NVL(v_avg, 0);
END;
```

Statement to Call from Flask (Back-end called by Front-end):

```
cursor.execute("SELECT fn_get_avg_rating(:p1) FROM DUAL", p1=user_id)
avg_rating = cursor.fetchone()[0]
```

3. Trigger: Auto-Update Project Progress

Oracle PL/SQL: This trigger ensures that when tasks are marked as 'Completed' on the front-end, the backend database scales the overall Project Idea progress up automatically up to 100%.

```
CREATE OR REPLACE TRIGGER trg_update_project_progress
AFTER UPDATE OF status ON tasks
FOR EACH ROW
WHEN (NEW.status = 'Completed')
DECLARE
    v_total     NUMBER;
    v_completed NUMBER;
    v_progress  NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_total FROM tasks WHERE idea_id = :NEW.
   idea_id;
    SELECT COUNT(*) INTO v_completed FROM tasks
           WHERE idea_id = :NEW.idea_id AND status = 'Completed';

    IF v_total > 0 THEN
        v_progress := ROUND((v_completed / v_total) * 100);
        UPDATE startup_ideas SET progress = v_progress WHERE idea_id =
   :NEW.idea_id;
    END IF;
END;
```

How it is fired from the Front-End: This trigger fires automatically whenever the user updates a task status to "Completed". The API executes the following table update, inciting the trigger:

```
cursor.execute("UPDATE tasks SET status = :p1 WHERE task_id = :p2", p1=
   "Completed", p2=task_id)
```

6. Screenshots of results generated after procedures and functions are called

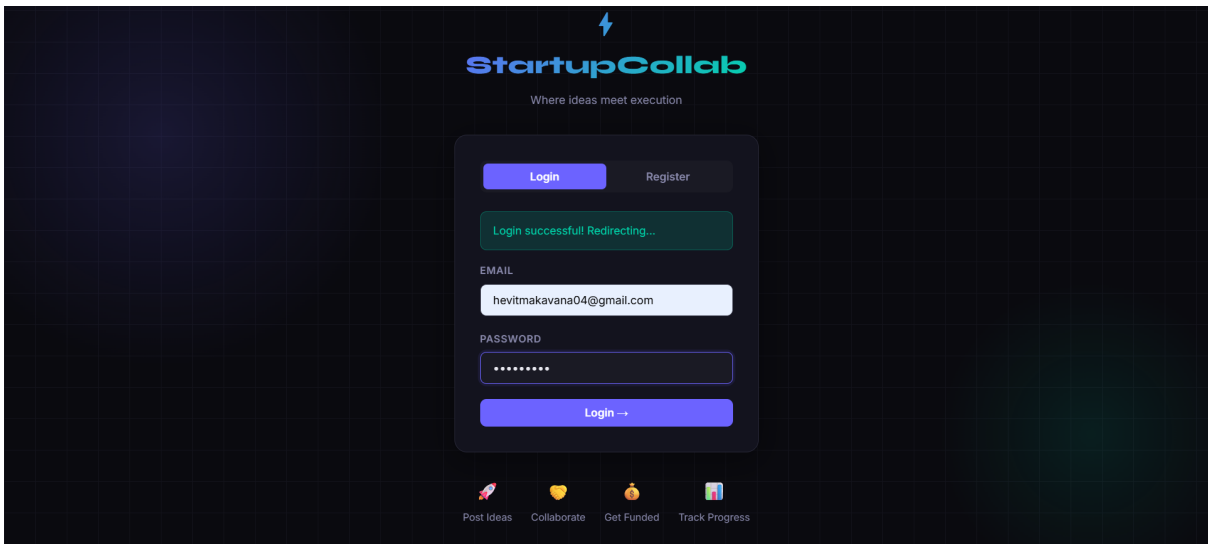


Figure 2: Successful Result on Front-End post-Procedure execution of Login Page

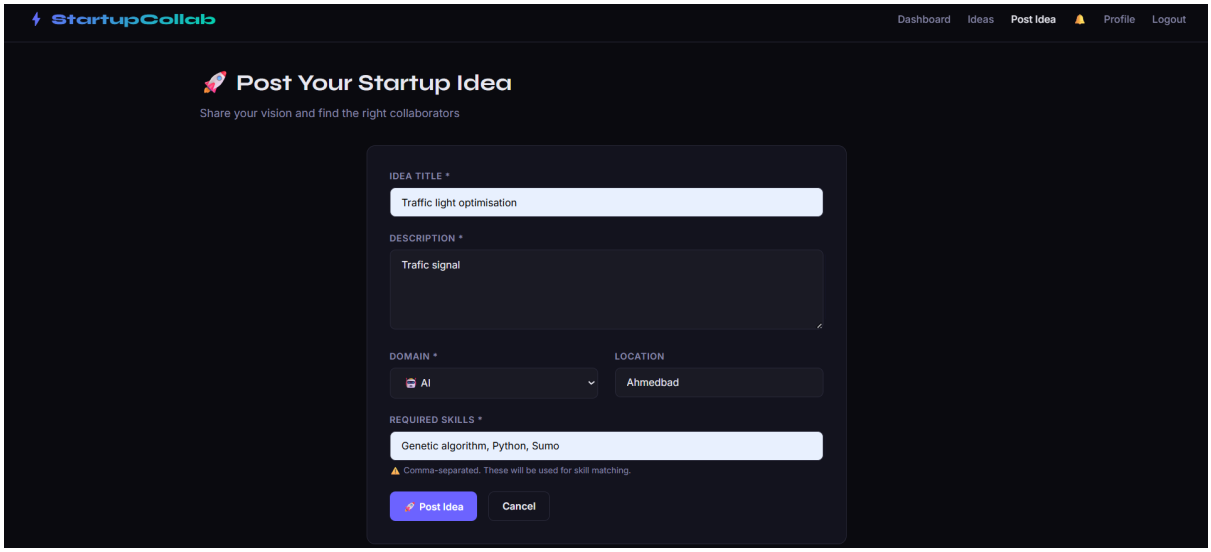


Figure 3: Successful Result on Front-End post-Procedure execution of Post Ideas

```
-- 8.4 TRIGGER: Auto-update project progress when task is completed
CREATE OR REPLACE TRIGGER trg_update_project_progress
AFTER UPDATE OF status ON tasks
FOR EACH ROW
WHEN (NEW.status = 'Completed')
DECLARE
    v_total     NUMBER;
    v_completed NUMBER;
    v_progress  NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_total FROM tasks WHERE idea_id = :NEW.idea_id;
    SELECT COUNT(*) INTO v_completed FROM tasks WHERE idea_id = :NEW.idea_id AND status = 'Completed';

    IF v_total > 0 THEN
        v_progress := ROUND((v_completed / v_total) * 100);
        UPDATE startup_ideas SET progress = v_progress WHERE idea_id = :NEW.idea_id;
    END IF;
END;
/
```

Figure 4: CODE screenshot

7. Screenshots of errors generated on the front-end when trigger is fired

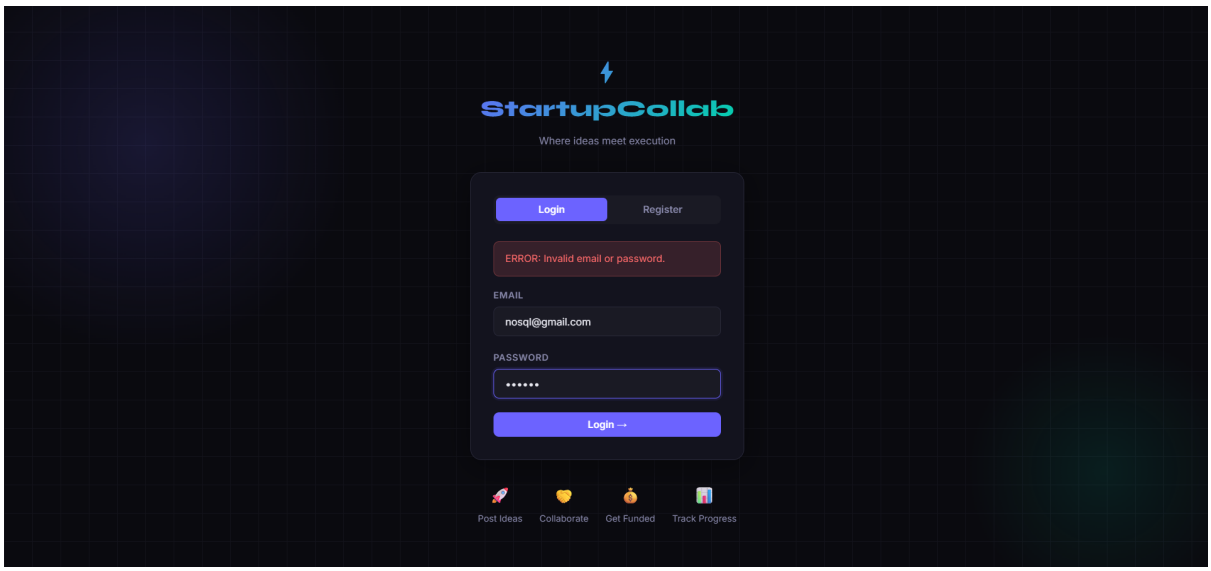


Figure 5: Errors generated if the login credentials is wrong

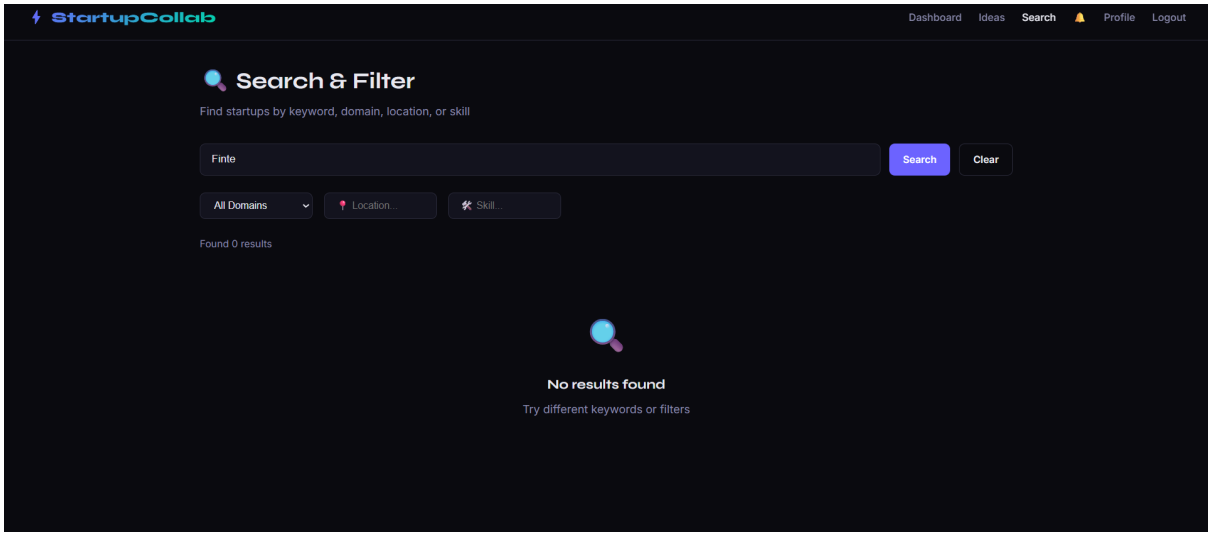


Figure 6: Search result if the idea does not exist

5. Individual Percentage Contribution

Roll No.	Name	Contribution
AU2340258	Harsh Patel	Frontend
AU2340194	Hevit Makavana	Backend
AU2340087	Maanya Patel	Database
AU2340055	Krissa Gandhi	Testing

Table 1: Student Contribution Chart